```
  except Exception as e:
    return jsonify({"error": str(e)}), 400


if __name__ == "__main__":
  app.run(debug=True)
```

**#1 Define the schema of the response.**
**#2 Define a single Query on the API.**
**#3 The Open Meteo API only works with coordinates, not city names, so we need to geocode the input city.**
**#4 Once we have the coordinates of the city, we can call the Open Meteo API.**

Once you are running the simple GraphQL server shown in listing 5.4 you can retrieve the forecast temperature and rainfall for the next 7 days for any city with a query as follows for Perth:

```
curl -X POST http://127.0.0.1:5000/graphql \
 -H "Content-Type: application/json" \
 -d '{
    "query": "{ weather(city: \"Perth\") { time rain surfaceTemperatu
re } }"
 }'
```

Listing 5.5 shows a fragment of the GraphQL schema of the first version of home digital twin API (the source code of the API is available in the GitHub repository). The API provides a query that you can use to look at a space over a period of time, and returns all information related to that space, for that time period. Following this approach, I do not need to know what specific sensors are measuring what parts of physical space (as I did in my example in section 5.1)—the API orchestrates retrieving relationships from the knowledge graph, querying source systems, and constructing a response.